Practical Set Pandas (II)

Page No:
Date / /

```
import pandas as pd
import numpy as np
```

Pandas Series: A series is a one-dimensional labeled array capable of holding any data type. The axis labels are collectively called the index.

Creating Series: There are multiple ways to create a series

```
labels = ['a', 'b', 'c']
my_list = [10, 20, 30]
arr = np.array([10, 20, 30])
dict = {'a': 10, 'b': 20, 'c': 30}

pd.Series(my_list, index=labels)

# a  10
  b  20
  c  30

pd.Series(arr, index=labels)

pd.Series(dict)
```

Dataframe: Creating a Dataframe

```
data = {'Name': ['John', 'Anna', 'Peter', 'Linda'],
        'Age': [28, 34, 29, 42],
        'City': ['New York', 'Paris', 'Berlin', 'London'],
        'Salary': [65000, 70000, 62000, 85000]}
```

Page No.:
Date: / /

```
df = pd.DataFrame(data)
print(df)
```

#	Name	Age	City	Salary
0	John	28	New York	65000
1	Anna	34	Paris	70000
2	Peter	29	Berlin	62000
3	Linda	42	London	85000

```
data_list = [['John', 28, 'New York', 65000],
             ['Anna', 34, 'Paris', 70000],
             ['Peter', 29, 'Berlin', 62000],
             ['Linda', 42, 'London', 85000]]

columns = ['Name', 'Age', 'City', 'Salary']
df2 = pd.DataFrame(data_list, columns = columns)
df2
```

#	Name	Age	City	Salary
0	John	28	New York	65000
1	Anna	34	Paris	70000
2	Peter	29	Berlin	62000
3	Linda	42	London	85000

```
#Selection and indexing of columns

df2[['Name', 'City']] # Selecting the column

#Creating a new column
```

Page No:
Date / /

```
df2['Designation'] = ['Manager', 'Eng.', 'Manager', 'Eng.']
```

#Removing Columns

```
df2.drop('Designation', axis=1, inplace=True) #We have to use axis=1 for Columns (horizontal) and axis=0 for rows or index (vertical). The inplace=True help us to implement this function to permanently delete the row or column from the DataFrame.

df2.drop(['City', 'Salary'], axis=1)
```

#Selecting Rows

```
df2.loc[0] #.loc function used to accessing or selecting the rows only by their character will see this in later.
df2.loc[[0, 1]]

#with .iloc
df2.iloc[3] #.iloc function used to accessing or selecting the rows only by their index its pretty seems like .loc functions but both is different
df2.iloc[[1, 2]]
```

#Selecting subsets of Rows and Columns

```
#Here I want to Select this portion:[
# [New York        65000]
  [Paris          70000]]
```

Page No.:
Date: / /

```
df2.loc[[0, 1]][['City', 'Salary']]

#Now the challange is to select this portion:
#Here we have to Select this portion:[
#[Peter          29]
  [Linda         42]]
df2.iloc[[2, 3]][['Name', 'Age']]
```

#Conditional Selection

#What are conditions? Conditions are generally Greater than (>), Greater than or equal to (>=), Less than (<), Less than or equal to (<=), Equal to (==), and not Equal to (!=).

```
df3=df2[df2['Age']>30]

#Get the data of only those people whose age is
  greater than 30 and lives in Paris.

df3=df2[(df2[df2['Age']>30]) & (df2['City']=='Paris')]
```

#Missing Data

#Finding Missing Data

```
data={
    'A':[1, 2, np.nan, 4, 5],
    'B':[np.nan, 2, 3, 4, 5],
    'C':[1, 2, 3, np.nan, np.nan],
    'D':[1, np.nan, np.nan, np.nan, 5]
    }
```

```
df = pd.DataFrame(data)
pd.isna(df)
```

#	A	B	C	D
0	False	True	False	False
1	False	False	False	True
2	True	False	False	True
3	False	False	True	True
4	False	False	True	False

```
df.isna().sum() #Returns the total number of nan values
```

#	
A	1
B	1
C	2
D	3

```
df.isna().any() #Returns the Boolean that any nan Value exist or not in the column.
```

#	
A	True
B	True
C	True
D	True

#Removing missing data

df

#	A	B	C	D
0	1	NaN	1.0	1.0
1	2	2.0	2.0	NaN
2	3	3.0	3.0	NaN
3	4	4.0	NaN	NaN
4	5	5.0	NaN	5.0

Page No.:
Date: / /

df.dropna() #Removes all columns which have at least one null value.

#With a new list where there is no any null value in 2 columns.

```
data={
    'A':[1, 2, 3, 4, 5],
    'B':[1, 2, 3, 4, 5],
    'C':[1, 2, 3, 4, 5],
    'D':[1, np.nan, np.nan, np.nan, 5]
    }

df=pd.Dataframe(data)

df.dropna() #Remove all the columns which have null values.
```

#	A	B	C	D
0	1	1	1	1.0
4	5	5	5	5.0

df.dropna(thresh=4) #In thresh=n parameter this returns the column which has not at least n number of not null values.

#Filling the missing data

#_#Now we will fill the missing data

df

#	A	B	C	D
0	1	1	1	1.0
1	2	2	2	NaN
2	3	3	3	NaN
3	4	4	4	NaN
4	5	5	5	5.0

df.fillna(0)

#	A	B	C	D
0	1	1	1	1.0
1	2	2	2	0.0
2	3	3	3	0.0
3	4	4	4	0.0
4	5	5	5	5.0

values = {'A': 0, 'B': 100, 'C': 300, 'D': 400}
df.fillna(values) # Fill all the missing values column wise.

#	A	B	C	D
0	1	1	1	1.0
1	2	2	2	400.0
2	3	3	3	400.0
3	4	4	4	400.0
4	5	5	5	5.0

df.fillna(df.mean()) # fill the mean value in missing value.

Page No.:
Date: / /

#	A	B	C	D
0	1	1	1	1.0
1	2	2	2	3.0
2	3	3	3	3.0
3	4	4	4	3.0
4	5	5	5	5.0

Merging, Joining & Concatination

Merging 2 Dataframes

```
employees = pd.DataFrame({
    'employee_id': [1, 2, 3, 4, 5],
    'name': ['John', 'Anna', 'Peter', 'Linda', 'Bob'],
    'department': ['HR', 'IT', 'Finance', 'IT', 'HR']
    })

#Dataframe2: Salary Information
Salaries = pd.DataFrame({
    'employee_id': [1, 2, 3, 4, 5],
    'Salary': [60000, 80000, 65000, 70000, 90000],
    'bonus': [5000, 10000, 7000, 8000, 12000]
    })

pd.merge(employees, Salaries, on='employee_id') #On used
        to merge in the basis of given parameter.
```

#	employee_id	name	department	Salary	bonus
0	1	John	HR	60000	5000
1	2	Anna	IT	80000	10000

2	3	Peter	Finance	65000	7000
3	4	Linda	IT	70000	8000
4	5	Bob	HR	90000	12000

pd.merge(employees, salaries, on='employee_id', how='inner')
#"How=" parameter help to arrange all the data with the basis of "inner" or "outer" column.

#	employee_id	name	department	Salary	bonus
0	1	John	HR	60000	5000
1	2	Anna	IT	80000	10000
2	3	Peter	Finance	65000	7000
3	4	Linda	IT	70000	8000
4	5	Bob	HR	90000	12000

pd.merge(employees, salaries, on='employee_id', how='left')
#How parameter help to arrange all the data with the basis of 'left' or 'right' column.

#	employee_id	name	department	Salary	bonus
0	1	John	HR	60000	5000
1	2	Anna	IT	80000	10000
2	3	Peter	Finance	65000	7000
3	4	Linda	IT	70000	8000
4	5	Bob	HR	90000	12000

#Concatination of 2 Dataframes

```
df1 = pd.DataFrame({'A': ['A0', 'A1', 'A2'],
                    'B': ['B0', 'B1', 'B2'],
                    'C': ['C0', 'C1', 'C2']})
```

Page No.:
Date: / /

```
df2 = pd.DataFrame({'A': ['A3', 'A4', 'A5'],
                    'B': ['B3', 'B4', 'B5'],
                    'C': ['C3', 'C4', 'C5']})
```

df1

#	A	B	C
0	A0	B0	C0
1	A1	B1	C1
2	A2	B2	C2

df2

#	A	B	C
0	A3	B3	C3
1	A4	B4	C4
2	A5	B5	C5

pd.concat([df1, df2])

#	A	B	C
0	A0	B0	C0
1	A1	B1	C1
2	A2	B2	C2
0	A3	B3	C3
1	A4	B4	C4
2	A5	B5	C5

pd.concat([df2, df1])

#	A	B	C
0	A3	B3	C3
1	A4	B4	C4
2	A5	B5	C5
0	A0	B0	C0
1	A1	B1	C1
2	A2	B2	C2

pd.concat([df1, df2], axis = 1)

#	A	B	C	A	B	C
0	A0	B0	C0	A3	B3	C3
1	A1	B1	C1	A4	B4	C4
2	A2	B2	C2	A5	B5	C5

Joining of 2 DataFrames

Page No:
Date / /

```
df1 = pd.DataFrame({
    'name': ['Alice', 'Bob', 'Charlie']
}, index=[1, 2, 3])

df2 = pd.DataFrame({                    #Second Dataframe
    'score': [85, 90, 75]
}, index=[2, 3, 4])
```

df1

#	name
1	Alice
2	Bob
3	Charlie

df2

#	Score
2	85
3	90
4	75

```
df1.join(df2, how='Outer')
```

#	Name	score
1	Alice	NaN
2	Bob	85.0
3	Charlie	90.0
4	NaN	75.0

```
df2.join(df1, how='Outer')
```

#	Score	Name
1	NaN	Alice
2	85.0	Bob
3	90.0	Charlie
4	75.0	NaN

#Group by an Aggregation

Page No.:
Date: / /

Groupby

```
data = {
    'Category': ['A', 'B', 'A', 'B', 'A', 'B', 'A', 'B'],
    'Store': ['S1', 'S1', 'S2', 'S2', 'S1', 'S2', 'S2', 'S1'],
    'Sales': [100, 200, 150, 250, 120, 180, 200, 300],
    'Quantity': [10, 15, 12, 18, 8, 20, 15, 25],
    'Date': pd.date_range('2023-01-01', periods=8)
}

df = pd.DataFrame(data)

df
```

#	Category	Store	Sales	Quantity	Date
0	A	S1	100	10	2023-01-01
1	B	S1	200	15	2023-01-02
2	A	S2	150	12	2023-01-03
3	B	S2	250	18	2023-01-04
4	A	S1	120	8	2023-01-05
5	B	S2	180	20	2023-01-06
6	A	S2	200	15	2023-01-07
7	B	S1	300	25	2023-01-08

Group by Category and calculate the sum of Sales

```
cat = df.groupby('Category')['Sales'].sum()
for v, i in cat:
    print(v)
    print(i)
```

#	Category	Store	Sales	Quantity	Date
0	A	S1	100	10	2023-01-01
2	A	S2	150	12	2023-01-03
4	A	S1	120	8	2023-01-05
6	A	S2	200	15	2023-01-07

B

#	Category	Store	Sales	Quantity	Date
1	B	S1	200	15	2023-01-02
3	B	S2	250	18	2023-01-04
5	B	S2	180	20	2023-01-06
7	B	S1	300	25	2023-01-08

```
Cat = df.groupby('Category')['Sales'].sum()
Print(cat)
```

# Category	
A	570
B	930

```
#Groupby Store and calculate the sum of Sales
cat=df.groupby('Store')['Sales'].sum()
Print(cat)
```

# Store	
S1	720
S2	780

```
#Group by multiple columns
#Group by Category and Store
cat=df.groupby(['Category','Store'])['Sales'].sum()
Print(cat)
```

# Category	Store	
A	S1	220
	S2	350
B	S1	500
	S2	430

Page No.:
Date: / /

Aggregation

```
df['Sales'].mean()
# Mean, Median, Mode, Min, Max, Count, Std
df['Sales'].median()
df['Sales'].mode()
df['Sales'].min()
df['Sales'].max()
df['Sales'].count()
df['Sales'].std()

df['Sales'].agg(['sum', 'mean', 'min', 'max', 'count', 'std', 'median', 'average
"]) # for mode we use average...
```

# Sum	1500.000000
mean	187.500000
min	100.000000
max	300.000000
count	8.000000
std	66.06274
median	190.000000
average	187.500000